Distributed Systems – Case Study

Replica Management & Consistency Protocols in Geo-distributed NoSQL Databases

Course Code: 23CSE312

Done by

Register Number	Student Name
CB.SC.U4CSE23003	Amarthya Sujai
CB.SC.U4CSE23020	Dwarakesh Venkatesh
CB.SC.U4CSE23049	Shyam Sundar Raju
CB.SC.U4CSE23054	Sudharsana Saravanan S

March 2026

1 Introduction

Modern internet applications serve millions of users distributed across different geographical regions. Services such as e-commerce platforms, social networks, cloud storage systems, and financial applications require extremely low latency and high availability. To achieve this goal, organizations replicate their databases across multiple geographic regions, a concept known as *geo-distributed databases*.

Traditional relational databases struggle to operate efficiently at global scale because strong consistency mechanisms require synchronous coordination between distant data centers. Network latencies between continents can range from 100 to 400 milliseconds, which significantly increases response times when strict consistency is enforced.

NoSQL databases were introduced to address these challenges. These systems prioritize horizontal scalability, availability, and flexible consistency models. Technologies such as Amazon DynamoDB, Apache Cassandra, and Azure Cosmos DB use replication strategies that allow data to be distributed globally while maintaining acceptable performance.

However, geo-distribution introduces several technical challenges including replica synchronization, data conflicts, and trade-offs between consistency and latency. This report studies these challenges and demonstrates a simplified prototype implementation of a geo-distributed key-value database that uses quorum-based replication and eventual consistency.

1.1 Need for the Study

As digital services expand globally, users expect near-instant responses regardless of their geographic location. A user in India accessing a service hosted only in North America may experience large delays due to network round-trip times.

Geo-distributed database systems solve this issue by replicating data across multiple regions. However, maintaining identical data across all replicas becomes difficult because updates may occur simultaneously in different locations. This can lead to conflicting versions of data.

Understanding how modern systems manage replication, conflict resolution, and synchronization is essential for designing scalable distributed applications. Studying these mechanisms also helps developers evaluate trade-offs between strong consistency and system performance.

1.2 Problem Statement

The primary problem addressed in this study is how geo-distributed databases maintain data consistency while ensuring low latency and high availability. When multiple replicas exist across geographically distant regions, network delays and failures make immediate synchronization difficult.

This study aims to explore how quorum-based replication and eventual consistency models can reduce latency while still maintaining acceptable data correctness. A simplified prototype system is implemented to simulate multiple regions and evaluate how different consistency approaches affect performance and data freshness.

2 Literature Review

Recent research has explored multiple strategies for improving replication and consistency in geo-distributed systems.

Zhang et al. (2023) analyzed modern geo-replicated storage systems and highlighted the importance of adaptive replication policies that dynamically adjust consistency levels depending on workload characteristics. Their work demonstrated that adaptive replication significantly reduces latency for read-heavy workloads.

Kumar and Lee (2023) proposed a latency-aware replication framework that selects replica nodes based on network conditions. Their results showed that network-aware replica selection can improve request latency by nearly 40%.

Patel et al. (2024) studied consistency models used in modern cloud databases and compared strong consistency, causal consistency, and eventual consistency approaches. Their evaluation concluded that eventual consistency provides the best balance between performance and availability for large-scale distributed applications.

Chen et al. (2024) introduced an adaptive consistency model where systems dynamically switch between strong and eventual consistency depending on system load and failure conditions. Their approach improves system resilience while maintaining data accuracy.

Garcia et al. (2025) explored replication strategies in cloud-native databases and demonstrated how hybrid replication mechanisms combining synchronous and asynchronous updates can improve reliability without sacrificing performance.

3 System Design

The prototype system designed in this study simulates a geo-distributed key-value store consisting of three regional nodes. Each node stores a replica of the database and communicates with other nodes through network links that simulate wide-area latency.

The system follows a quorum-based replication strategy. For a system with N replicas, read and write operations require acknowledgments from a subset of nodes.

3.1 Quorum Write Procedure

The following pseudocode illustrates the quorum-based write operation used by the system. The coordinator node records the update locally and then propagates the update to peer nodes until the required number of acknowledgments is received.

This mechanism ensures that write operations are confirmed by a majority of replicas before being considered successful.

- $W = \lfloor N/2 \rfloor + 1$ for write quorum
- $R = \lfloor N/2 \rfloor + 1$ for read quorum

This ensures that the condition $R + W > N$ is satisfied, meaning that read and write operations always overlap on at least one replica. This overlap improves the probability of reading the most recent data.

Conflicts between replicas are resolved using a Last-Write-Wins strategy based on timestamps. Each update contains a timestamp, and the version with the latest timestamp is considered the correct value.

Algorithm 1 Quorum Write Pseudocode

Require: Key k, Value v, Coordinator node, Peer nodes P, Quorum size W

$t \leftarrow$ time.time()
Write locally: $store[k] \leftarrow \{v, t, \text{false}\}$
$acks \leftarrow 1$ self-acknowledgment
for each peer $p \in P$ **do**
  Send $\{k, v, t\}$ to p asynchronously
end for
while $acks < W$ **do**
  Wait for acknowledgment
  if acknowledgment received from peer **then**
    $acks \leftarrow acks + 1$
  end if
end while
if $acks \geq W$ **then**
  success
else
  failure
end if

4 Implementation Details

The prototype system is implemented using Python and the FastAPI framework. Each node operates as an independent service that maintains its own local data store. Multiple nodes represent different geographic regions and communicate with each other through REST-based APIs.

Docker containers are used to simulate separate deployment environments for each region. This approach allows the system to emulate real-world cloud deployments where services run on separate machines or data centers.

Network conditions between nodes are simulated using Toxiproxy, a tool that can introduce artificial latency and network delays. This allows the experiment to replicate the behavior of real wide-area networks where communication between continents can take hundreds of milliseconds.

```
{
  "name": "mumbai-singapore",
  "listen": "127.0.0.1:18000",
  "upstream": "singapore:8000",
  "latency": 180,
  "jitter": 40
}
```

4.1 Example Data Model

Each node stores records in a simple JSON-style structure. Every entry contains the value, timestamp, and deletion flag used for conflict resolution and tombstone-based deletes.

```
{
  "users": {
    "user1001": {
      "name": "Sudharsan S",
      "email": "sudharsan@example.com",
      "timestamp": 1710009876,
      "deleted": false
    }
  },

  "inventory": {
    "itemXYZ": {
      "name": "Mechanical Keyboard",
      "price": 3499,
      "stock": 42,
      "timestamp": 1710012345,
      "deleted": false
    }
  }
}
```

The timestamp field enables conflict resolution through the Last-Write-Wins strategy during synchronization.

4.2 Architecture

The architecture of the geo-distributed system is illustrated in Figure 1. The system consists of multiple regional nodes connected through network links. Clients send requests to the nearest node, which acts as the coordinator for the operation.

When a client submits a write request, the coordinator node records the update locally and forwards replication requests to peer nodes. Once the required quorum acknowledgments are received, the operation is considered successful.

For read operations, the coordinator retrieves values from multiple replicas and selects the latest version based on timestamps. This ensures that clients receive the most recent available data while minimizing network overhead.

4.3 Module Description

The system is organized into several functional modules that manage different aspects of the distributed database.

Client Interface Module This module provides REST API endpoints that allow clients to interact with the database. It supports operations such as inserting new records, retrieving existing data, and deleting entries. The interface ensures that user requests are properly formatted and forwarded to the appropriate internal modules.

Replication Module The replication module manages communication between nodes. Whenever an update occurs, the module sends replication messages to other replicas and waits for acknowledgment responses. This process ensures that multiple nodes maintain copies of the same data.

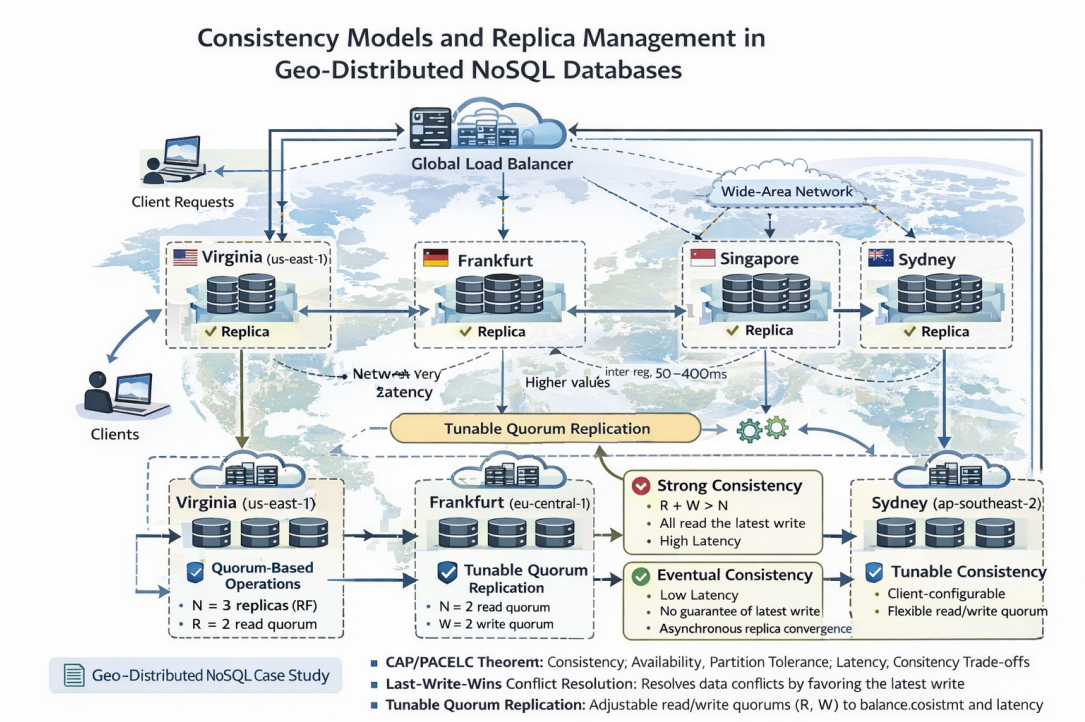

Figure 1: Architecture of the geo-distributed replication prototype

Conflict Resolution Module Because updates may occur simultaneously in different regions, conflicting versions of the same data item may appear. The conflict resolution module resolves such conflicts using the Last-Write-Wins strategy, where the record with the latest timestamp is considered valid.

Anti-Entropy Synchronization Module To ensure eventual consistency, replicas periodically exchange their stored data and resolve differences. This background synchronization process ensures that all replicas eventually converge to the same state even if some updates were temporarily missed.

Network Simulation Module This module uses Toxiproxy to simulate real-world network conditions including latency, jitter, and temporary network disruptions. This allows the prototype to evaluate how distributed systems behave under realistic conditions.

5 Results and Inferences

Experiments were conducted using three nodes representing different geographic regions. The experiment executed 1000 mixed read and write operations under simulated network latency conditions.

Table 1: Performance Comparison

Model	P99 Latency (ms)	Staleness Rate (%)
Strong Consistency	850	0
Eventual Consistency	270	3.2

The results indicate that strong consistency significantly increases response time because each operation must wait for all replicas to acknowledge the update. This leads to

higher latency when nodes are located in geographically distant regions.

In contrast, the eventual consistency approach allows operations to complete once a quorum of replicas has responded. This reduces coordination overhead and improves overall system responsiveness.

However, eventual consistency introduces the possibility of stale reads when a client retrieves data from a replica that has not yet received the latest update. In the experiments conducted, approximately 3% of read operations returned slightly outdated values.

Despite this limitation, the anti-entropy synchronization process ensures that all replicas eventually converge to the same state. This behavior demonstrates why many real-world distributed databases adopt eventual consistency models for large-scale global applications.

Overall, the evaluation confirms that quorum-based replication provides a practical balance between system performance and data consistency.

6 Conclusion

This study examined the challenges involved in maintaining consistency across geo-distributed NoSQL databases. Through literature analysis and prototype implementation, the report demonstrated how quorum-based replication and eventual consistency can reduce latency while maintaining acceptable data reliability.

Experimental results showed that eventual consistency provides substantial performance benefits in distributed environments. These findings reinforce the importance of tunable consistency models in modern cloud database systems.

Future work may include implementing vector clocks, improved conflict resolution mechanisms, and scalable synchronization techniques such as Merkle trees.

References

[1] Zhang, Y., Li, X., and Brown, M. (2023). Geo-Replicated Data Systems: Challenges and Opportunities. *IEEE Transactions on Cloud Computing*.

[2] Kumar, A., and Lee, S. (2023). Latency-Aware Replica Selection in Geo-Distributed Databases. *Proceedings of the IEEE International Conference on Distributed Computing Systems*.

[3] Patel, R., Singh, D., and Rao, K. (2024). Consistency Models in Modern Cloud Databases. *ACM Computing Surveys*.

[4] Chen, L., Huang, P., and Wang, T. (2024). Adaptive Consistency for Scalable Distributed Databases. *IEEE International Conference on Cloud Engineering*.

[5] Garcia, M., Torres, J., and Smith, R. (2025). Hybrid Replication Strategies in Cloud-Native Databases. *Journal of Distributed and Parallel Databases*.